ЛАБОРАТОРНАЯ РАБОТА №1

Блочное умножение матриц и векторизация

Теория, устройство кода и результаты на Apple M5

> Работа реализует умножение двух блочных матриц тремя способами — скалярным, с автоматической векторизацией и с ручной векторизацией на интринсиках SSE2 — и доказывает, что векторизованные версии дают **побитово одинаковый** результат при двукратном ускорении.
>
> Особенность решения: целевая машина построена на ARM64, где инструкций SSE2 физически не существует. Код тем не менее написан на настоящих интринсиках SSE2 и транслируется в NEON на этапе компиляции.

ТИП ДАННЫХ

double (64 бита, IEEE 754)

РАЗМЕР БЛОКА

10 × 10 элементов

ВНЕШНИЙ РАЗМЕР

400 × 400 блоков (4000 × 4000)

ЦЕЛЕВАЯ МАШИНА

Apple M5, ARM64, macOS

КОМПИЛЯТОР

Apple clang 21, C11, −O3 (Release)

SIMD

SSE2-интринсики → NEON (sse2neon)

Содержание

1	Постановка задачи	формальные требования и запреты
2	Целевая машина и главная сложность	ARM64 без SSE2, sse2neon
3	Векторизация и SIMD	наборы инструкций, условия автовекторизации
4	Подсистема памяти	иерархия, кэш-линия, TLB, предвыборка
5	Почему наивное умножение медленное	обход по столбцу, арифметическая интенсивность
6	Блочность и раскладка памяти	block-major, что это даёт
7	Ядро: схема Intel AP-930	умножение без транспонирования
8	Побитовая идентичность C1 и C2	неассоциативность, запрет FMA
9	Измерение времени	почему запрещены clock() и time()
10	Устройство проекта и разбор кода	файлы, контракты, ключевые функции
11	Результаты измерений	тайминги, GFLOP/s, ускорение
12	Доказательство векторизации	дизассемблер, отчёты, SLP
13	Особенности решения	что здесь нестандартного
14	Вопросы на защиту	короткие ответы

1 Постановка задачи

> Перемножить две блочные матрицы, где каждый элемент внешней матрицы — сам матрица меньшего размера, и сделать это тремя способами, доказав идентичность результатов и выигрыш от векторизации.

Формально: **A** состоит из $L \times M$ блоков размера $l \times m$, **B** — из $M \times N$ блоков размера $m \times n$, результат **C** — из $L \times N$ блоков размера $l \times n$:

$$C_{ij} = \Sigma_{k=0..M-1} A_{ik} \cdot B_{kj}$$

где A_{ik}, B_{kj} — подматрицы, а произведение и сумма — обычные матричные операции. Преподавателем зафиксированы тип элемента `double` и размер блока 10×10; внешний размер подбирается самостоятельно так, чтобы время построения C1 составляло несколько секунд.

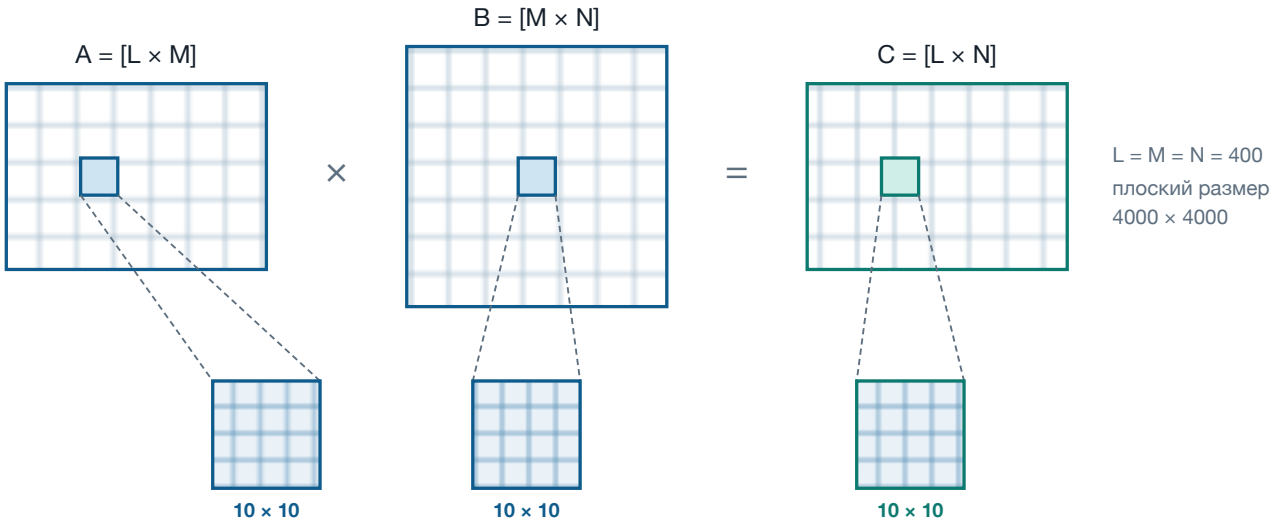


Рис. 1. Структура задачи: элемент внешней матрицы — блок 10 × 10 значений `double`.

Обязательные требования и запреты

ТРЕБОВАНИЕ	КАК ВЫПОЛНЕНО
Минимум две функции: автовекторизация (C1) и ручная векторизация SSE2 (C2)	`multiply_blocks_auto` и `multiply_blocks_sse2`; плюс третья, скалярная, как база сравнения
C1 и C2 должны совпадать **полностью**	сравнение `memcmp` по всему массиву, побитово, без epsilon
SSE2-версия не медленнее автовекторизованной	проверяется программой, печатается вердикт
Запрещено транспонировать матрицы	схема Intel AP-930 читает строки B вперёд, столбцы не нужны

ТРЕБОВАНИЕ	КАК ВЫПОЛНЕНО
Запрещено выводить фрагменты матриц	печатаются только вердикты, тайминги и конфигурация
Запрещены `clock()` и `time()`	`clock_gettime(CLOCK_MONOTONIC_RAW)` и аппаратный счётчик
Только Release	`-O3`, единственная конфигурация в Makefile
Время построения C1 — несколько секунд	внешний размер подбирается замером

2 Целевая машина и главная сложность

ПАРАМЕТР	ЗНАЧЕНИЕ
Процессор	Apple M5, архитектура ARM64 (AArch64)
Ядер	10
L1d на ядро	`64 KiB`
L2	`6 MiB`
Размер кэш-линии	`128 байт` (не 64, как на x86)
SIMD	NEON, 128 бит, регистры v0–v31
Компилятор	Apple clang 21, C11

ПРОБЛЕМА

Задание требует ручной векторизации на **SSE2**. SSE2 — это набор инструкций x86. На ARM64 его не существует: нет ни регистров `xmm`, ни инструкций `mulpd`, ни интринсика `__rdtsc()`.

Решение: sse2neon

`sse2neon.h` — header-only библиотека, которая определяет типы `__m128`, `__m128d`, `__m128i` и функции `_mm_*` через типы и интринсики NEON. Трансляция происходит **на этапе компиляции**, один в один, без рантайм-слоя:

ПИШЕМ (SSE2)	РАЗВОРАЧИВАЕТСЯ В (NEON)	ИНСТРУКЦИЯ ARM64
`__m128d`	`float64x2_t`	регистр `v0..v31`
`_mm_loadu_pd`	`vld1q_f64`	`ldr q0, [x1]`
`_mm_set1_pd`	`vdupq_n_f64`	`dup v0.2d, x1`
`_mm_mul_pd`	`vmulq_f64`	`fmul.2d`
`_mm_add_pd`	`vaddq_f64`	`fadd.2d`

Семантика совпадает точно, потому что совпадает главное: **ширина регистра 128 бит и тип элемента**. И SSE2 `__m128d`, и NEON `float64x2_t` содержат ровно два `double`. Значит число дорожек, порядок элементов и правила округления IEEE 754 одинаковы — результат вычислений идентичен тому, что дал бы настоящий x86.

```c
/* Исходник честен относительно обеих платформ */
#if defined(__x86_64__) || defined(__i386__)
#include <emmintrin.h>       // настоящий SSE2
#elif defined(__aarch64__)
#include "sse2neon.h"        // трансляция в NEON
#else
#error "multiply_sse2.c requires SSE2 (x86) or sse2neon on aarch64"
#endif
```

ПОЧЕМУ ЭТО СИЛЬНЫЙ АРГУМЕНТ

Файл `multiply_sse2.c` написан на буквальных интринсиках SSE2 и ни в одном месте не использует NEON напрямую. Перенесённый на машину с x86, он соберётся через `<emmintrin.h>` **без единой правки** и будет выполнять настоящие инструкции `mulpd` / `addpd`. Условная компиляция в шапке — это весь объём платформенной зависимости.

Ограничение, о котором нужно знать: `__m128d` реализован в sse2neon только на **aarch64** — на 32-битном ARM в NEON вообще нет операций с двойной точностью. Поэтому условная компиляция выше не декоративна, а несёт нагрузку, и ветка `#else` намеренно обрывает сборку, вместо того чтобы молча собрать что-то неверное.

Счётчик тактов sse2neon не покрывает — это не SIMD. Его пришлось реализовать самостоятельно (см. §9).

3 Векторизация и SIMD

SIMD (Single Instruction, Multiple Data) — одна инструкция обрабатывает целый вектор данных. В классификации Флинна это отдельный класс наряду с SISD, MISD и MIMD.

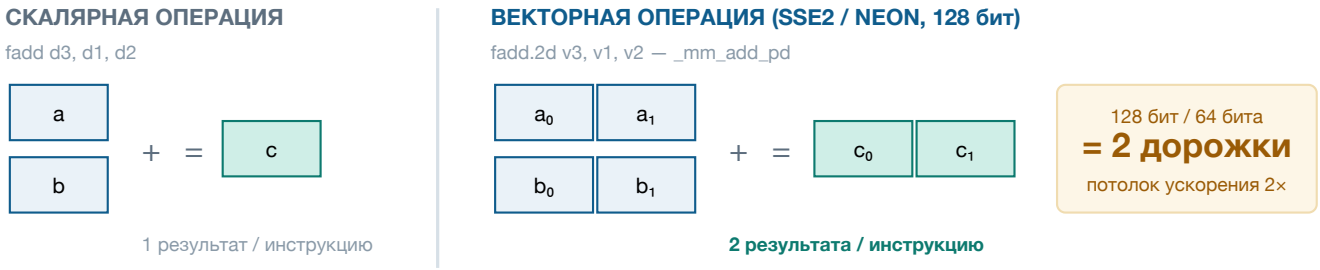


Рис. 2. Для `double` в 128-битный регистр помещается две величины, поэтому теоретический предел ускорения — двукратный.

Наборы SIMD-инструкций x86

НАБОР	ГОД	ШИРИНА	РЕГИСТРЫ	ТИПЫ ДАННЫХ
MMX	1997	64	mm0–mm7 *(алиас на стек x87)*	только целые
SSE	1999	128	xmm0–xmm15	float
SSE2	**2000**	**128**	**xmm0–xmm15**	**+ double, + целые**
AVX	2011	256	ymm	float, double
AVX2	2013	256	ymm	+ целые, + FMA
AVX-512	2016	512	zmm	+ регистры масок

Для этой работы важно, что **SSE2 — первый набор, поддерживающий** `double`. Отсюда `VECTOR_LANES = 16 / sizeof(double) = 2` и потолок ускорения 2×. Для `float` дорожек было бы четыре, и методичка соответственно называет ориентир ~3–3,5× для float против ~1,2–1,5× для double.

Историческая деталь, о которой спрашивают: регистры MMX физически совпадали со стеком сопроцессора x87, поэтому смешивать MMX- и FPU-код без инструкции `emms` было нельзя. SSE исправил это отдельным регистровым файлом.

Условия автоматической векторизации

Компилятор векторизует цикл, только если может **доказать** безопасность преобразования.

УСЛОВИЕ	ПОЯСНЕНИЕ
Итерации независимы	Нет зависимости, переносимой циклом. `a[i] = a[i−1] + b[i]` не векторизуется: значение следующей итерации зависит от предыдущей.

УСЛОВИЕ	ПОЯСНЕНИЕ
Число итераций вычислимо	Счётное количество итераций должно быть известно до входа в цикл. `while (p->next)` — нет.
Нет нарушения потока команд	Ни `break`, ни `goto`, ни вызовов невстраиваемых функций, ни исключений: один вход, один выход.
Внутренний цикл	Векторизуется, как правило, самый внутренний цикл гнезда.
Предсказуемый доступ	В идеале единичный шаг. Косвенная индексация `a[idx[i]]` требует gather/scatter, которых в SSE2 нет вообще.
Нет алиасинга	Главная практическая помеха. См. ниже.

Алиасинг — почему он решает всё

По правилам C любые два указателя `double*` могут указывать на перекрывающуюся память. Компилятор не знает, что запись в `result_row` не испортит чтение из `right_row`, и поэтому либо отказывается векторизовать, либо вставляет проверку перекрытия во время выполнения и порождает две версии цикла.

Решение — ключевое слово `restrict` из C99: обещание программиста, что в области видимости указателя доступ к объекту идёт только через него. В нашем коде это оказалось решающим:

```
/* restrict стоит на ПАРАМЕТРАХ — именно там компилятор его использует,
   чтобы доказать независимость итераций. */
static inline void accumulate_scaled_row(
        matrix_element *const restrict result_row,
        const matrix_element *const restrict right_row,
        const matrix_element left_value)
{
    for (int inner_col = 0; inner_col < BLOCK_COLS; ++inner_col) {
        result_row[inner_col] += left_value * right_row[inner_col];
    }
}
```

ПРАКТИЧЕСКОЕ НАБЛЮДЕНИЕ

Первая, «черновая» версия ядра без выделения этой функции давала около **4,8 GFLOP/s**. После того как обновление строки было вынесено в отдельную функцию с `restrict` на параметрах, та же арифметика стала выполняться на **33 GFLOP/s** — почти в семь раз быстрее. Разница целиком в том, смог ли компилятор доказать отсутствие алиасинга.

Dregper отмечает ровно эту причину, объясняя, почему Fortran до сих пор предпочитают для численных расчётов: там алиасинг запрещён семантикой языка, и компилятору не нужно ничего доказывать.

Как отключить векторизацию

Это прямой вопрос из методички. Флаги по компиляторам:

КОМПИЛЯТОР	ОТКЛЮЧЕНИЕ
clang	`-fno-vectorize -fno-slp-vectorize`; на цикл — `#pragma clang loop vectorize(disable) interleave(disable)`
GCC	`-fno-tree-vectorize -fno-tree-slp-vectorize`
MSVC	`#pragma loop(no_vector)`

ПОДВОХ, КОТОРЫЙ ЗДЕСЬ И ПРОЯВИЛСЯ

Векторизаторов **два**. Loop-векторизатор преобразует цикл. **SLP-векторизатор** (Superword Level Parallelism) работает иначе: он склеивает независимые скалярные операции в подряд идущем коде, вообще без цикла — например, после того как короткий цикл был полностью развёрнут.

В нашем случае весь векторный код порождён именно SLP-векторизатором, и флаг `-fno-vectorize` сам по себе не даёт ничего. Экспериментальное подтверждение — в §12.

4 Подсистема памяти

Процессор быстрее оперативной памяти примерно на два порядка, и разрыв рос десятилетиями: производительность ядра увеличивалась приблизительно на 50 % в год, а задержка DRAM снижалась лишь на ~7 %. Это явление называют **memory wall**. Кэш — способ его обойти.

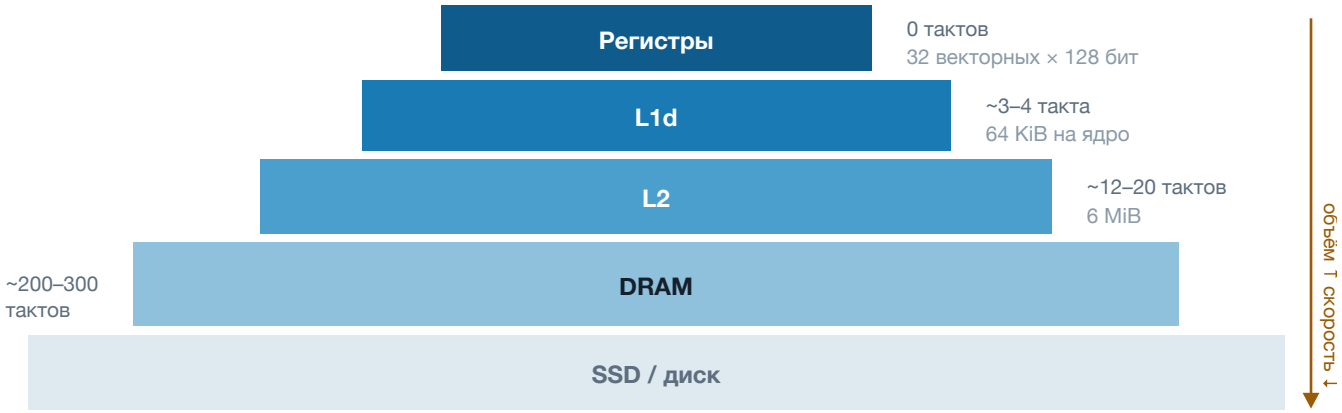


Рис. 3. Иерархия памяти целевой машины. Величины L1d/L2 и размер линии получены через `sysctl hw.*`.

Иерархия работает только благодаря **локальности** обращений:

Временна́я локальность

к одному и тому же адресу обратятся снова в ближайшее время — поэтому имеет смысл его удерживать

Пространственная локальность

к соседним адресам обратятся вскоре — поэтому имеет смысл подтягивать их заранее

Кэш-линия

Память передаётся между уровнями иерархии не байтами, а **линиями**. На x86 это обычно 64 байта, **на Apple M5 — 128 байт**. Последствия:

- Чтение одного `double` (8 байт) втягивает всю линию — 16 значений. Использовали все шестнадцать — отлично; использовали одно — 94 % трафика потрачено впустую.
- Отсюда правило Dreppер'а `SM = CLS / sizeof(T)`: разворачивать цикл ровно настолько, чтобы полностью выбрать линию.
- **False sharing** — в многопоточном коде два ядра пишут в разные переменные, попавшие в одну линию, и линия начинает «пинг-понговать» между кэшами. У нас однопоточно, но вопрос вероятен.

Ассоциативность и конфликтные промахи

Кэш обычно *N-way set associative*: адрес отображается в набор (set) по средним битам, внутри набора есть N мест. Если программа циклически обращается к N+1 адресам, попадающим в один набор, они вытесняют друг друга — **конфликтный промах**, при том что кэш в целом почти пуст.

Классическая ловушка матричного умножения: размерность, равная степени двойки. При обходе столбца матрицы $N \times N$ шаг равен `N · sizeof(T)` байт; если он кратен произведению числа наборов на размер линии, весь столбец укладывается в один набор. Отсюда характерные провалы производительности ровно на 1024, 2048 и так далее. Лечится дополнением строк (`N+1`) или блочной раскладкой — у нас работает второе.

Три канонических типа промахов (правило «3C»): **compulsory** — первое обращение к данным, **capacity** — рабочее множество не поместилось, **conflict** — не хватило ассоциативности.

TLB

Виртуальные адреса транслируются в физические через таблицы страниц; кэш этих трансляций — **TLB**. Страница обычно 4 KiB, а TLB первого уровня содержит лишь десятки записей.

Для матрицы 4000×4000 из `double` одна строка занимает 32 000 байт, то есть **почти каждая строка лежит на своей странице**. Обход по столбцу трогает 4000 разных страниц подряд — TLB переполняется, и каждая трансляция превращается в поход по таблицам страниц. Это отдельная, часто забываемая причина катастрофичности столбцового обхода — помимо промахов кэша.

Предвыборка

Аппаратная

детектор потоков распознаёт постоянный шаг и подтягивает линии заранее. Работает на предсказуемых линейных паттернах; бесполезна на обходе столбца большой матрицы и на указательных структурах.

Программная

`__builtin_prefetch`, `_mm_prefetch` — явная подсказка процессору.

Non-temporal stores (`_mm_stream_pd`, `movntpd`) — запись мимо кэша. Полезны, когда результат записывается и в ближайшее время читаться не будет, чтобы не вытеснять из кэша полезные данные. *В нашей задаче не применимы:* блок результата перечитывается на каждой итерации внешнего цикла по k, то есть обладает сильной временно́й локальностью — обходить кэш было бы прямо вредно.

Политика записи на современных процессорах — **write-back** с **write-allocate**: грязная линия записывается в память при вытеснении, а запись в отсутствующую линию сначала её читает (RFO, Read For Ownership). Поэтому запись в память стоит ещё и чтения — факт, который легко упустить при подсчёте трафика.

5 Почему наивное умножение медленное

```
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      c[i][j] += a[i][k] * b[k][j];   // b обходится ПО СТОЛБЦУ
```

Обращение `a[i][k]` идёт вдоль строки — шаг 1, всё хорошо. Обращение `b[k][j]` идёт **вдоль столбца** — шаг `N · 8` байт. Каждое такое обращение втягивает новую кэш-линию, из 16 загруженных `double` используется один, и попутно задевается новая страница памяти, то есть добавляется промах TLB.

Арифметическая интенсивность — отношение числа операций к объёму трафика памяти — ключевая метрика roofline-модели. У наивной версии на два флопа приходится порядка 16 байт загрузки:

2 флопа / 16 байт = ***0,125 флоп/байт***

Это глубоко в области, ограниченной памятью: процессор простаивает в ожидании данных, и его пиковая вычислительная мощность не имеет значения.

Три классических способа лечения

ПРИЁМ	СУТЬ И ОГРАНИЧЕНИЕ	ВРЕМЯ (DREPPER)
Исходная версия	Обход `b` по столбцу	`100 %`
Перестановка циклов *i-k-j*	Внутренним становится `c[i][j] += a_ik · b[k][j]` по *j*: и `b`, и `c` идут вдоль строк с шагом 1. Уже заметно лучше и хорошо векторизуется. Но при каждом *i* прочёсывается вся матрица `b` целиком.	—
Транспонирование `b`	Столбец превращается в строку. Требует дополнительной памяти под копию и самого прохода копирования. **В этой работе запрещено.**	`23,4 %`
Блочность (tiling)	**Разбиение на подматрицы, помещающиеся в L1; блок обсчитывается целиком, пока находится в кэше. Дополнительной памяти не требует.**	**`17,3 %`**
Блочность + векторизация	**То же плюс SIMD-ядро**	**`9,47 %`**

Цифры в последнем столбце — таблица 6.2 из работы Drepper «What Every Programmer Should Know About Memory». В абсолютных величинах это переход с 318 MFLOPS на 3,35 GFLOPS на одном ядре: ускорение более чем в десять раз **без единого изменения в алгоритме** — изменился только порядок обращения к памяти.

ВЫВОД, РАДИ КОТОРОГО ДЕЛАЕТСЯ ЭТА РАБОТА

Асимптотика у всех перечисленных версий одинакова — $\Theta(N^3)$, и число арифметических операций совпадает до единицы. Десятикратная разница во времени объясняется исключительно тем, насколько эффективно код использует кэш и насколько компилятор сумел задействовать SIMD.

6 Блочность и раскладка памяти

Задание сформулировано через блочные матрицы, и это не случайная декорация: блочное умножение — в точности **тайлинг**, зашитый прямо в постановку. Математически оно тождественно обычному умножению, поскольку

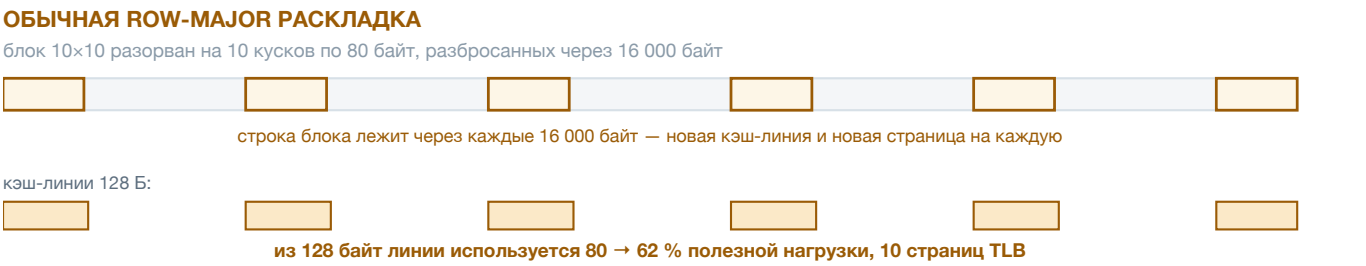
$$C_{ij} = \Sigma_k A_{ik} B_{kj}$$

справедливо и для подматриц. Иначе говоря, преподаватель заставляет реализовать кэш-оптимальную версию, сформулировав её как условие задачи.

Block-major: как блоки лежат в памяти

Ключевое проектное решение работы — **блочно-мажорная раскладка**. Блоки хранятся подряд, один за другим; внутри блока элементы идут по строкам. Адрес начала блока (i, j):

```
elements + (block_row * block_cols + block_col) * BLOCK_ELEMENT_COUNT
```



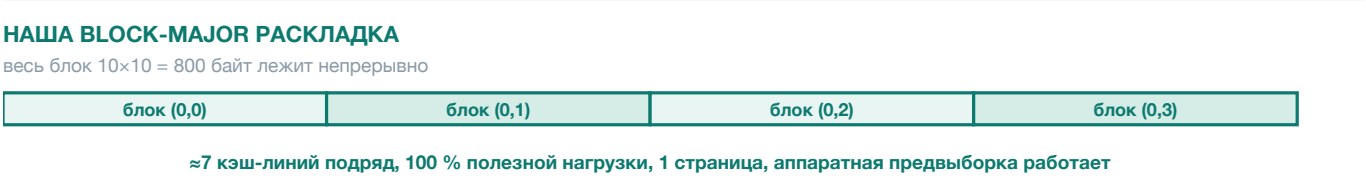


Рис. 4. Один и тот же блок 10 × 10 при обычной и при блочной раскладке.

Что это даёт

СВОЙСТВО	СЛЕДСТВИЕ
Блок = 800 байт	Три блока (A, B, C) — 2400 байт. Против L1d в 64 KiB это запас в 27 раз: умножение пары блоков идёт целиком из кэша первого уровня.
Блок непрерывен	Занимает ≈7 кэш-линий подряд, каждая используется на 100 %. При обычной раскладке блок был бы разорван на 10 строк по 80 байт из 128.
Одна страница вместо десяти	800 байт против 4 KiB страницы — блок гарантированно занимает одну–две страницы, а не десять. Давление на TLB падает на порядок.

СВОЙСТВО	СЛЕДСТВИЕ
Нет большого шага	Конфликтные промахи исчезают: мы нигде не обходим память с шагом, кратным размеру набора.
Линейный доступ	Аппаратная предвыборка распознаёт поток и подтягивает данные заранее.
Выравнивание 128 байт	Буфер выровнен на размер кэш-линии целевой машины, поэтому блоки не пересекают границы линий лишний раз.

Арифметическая интенсивность блочной версии: на пару блоков приходится $2 \cdot 10^3 = 2000$ флопов и около 2400 байт трафика:

2000 флопов / 2400 байт ≈ ***0,83 флоп/байт*** *— почти в 7 раз лучше наивной*

Порядок внешних циклов

Выбран порядок *i–k–j*: блок `A[i][k]` фиксируется и переиспользуется на всём проходе по *j*, оставаясь в регистрах и L1, а блоки B и C читаются последовательно. Это регистровое блокирование, надстроенное над кэшевым.

ЧЕСТНАЯ ОЦЕНКА ГРАНИЦ РЕШЕНИЯ

При L = 400 внешний цикл прочёсывает всю матрицу B (122 МБ, в L2 объёмом 6 МБ она не помещается) ровно L раз — это порядка 51 ГБ трафика из DRAM. Однако считаем мы 128 GFLOP примерно за 3,8 с, то есть требуемая полоса составляет около 13,5 ГБ/с — величина, комфортная для M5. Значит **мы упираемся в вычисления, а не в память**.

Если бы понадобилось ускоряться дальше, следующими шагами были бы второй уровень блокировки под размер L2, упаковка панелей (packing) и использование FMA — то есть путь, которым идёт GotoBLAS.

7 Ядро: схема Intel AP-930

Запрет транспонирования — центральное ограничение работы, и обойти его «в лоб» не получается. Наивное векторное умножение хочет вычислить скалярное произведение строки A на столбец B, но столбец лежит в памяти с шагом и одной векторной загрузкой не берётся. Отсюда и возникает соблазн транспонировать.

Решение, описанное в документе Intel AP-930 «Streaming SIMD Extensions — Matrix Multiplication», состоит в том, чтобы **перевернуть задачу**: вместо «строка × столбец → одно число» вычислять «**один скаляр × целая строка → накопление в строку результата**».

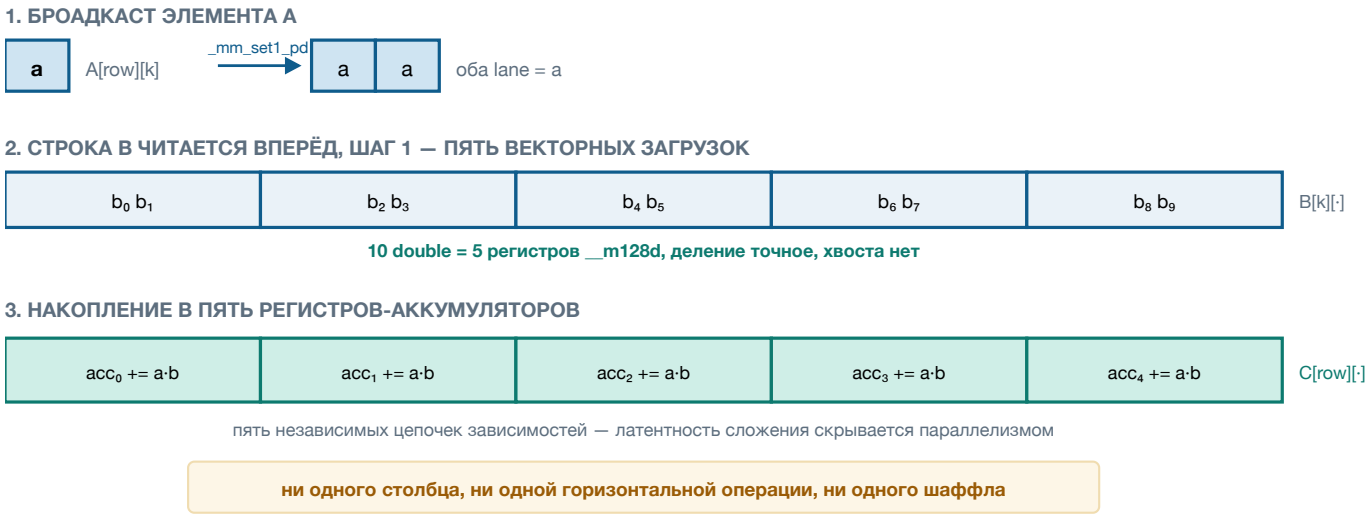


Рис. 5. Broadcast-and-accumulate: элемент A размножается по дорожкам и умножается на строку B.

Intel формулирует принцип так: «каждая строка результата зависит от всех строк одной матрицы, но только от одной строки другой» — поэтому выгодно держать накопители в регистрах, а элементы второго операнда подгружать по одному.

Почему это правильный ответ на запрет транспонирования

- **Строка B читается вперёд с шагом 1.** `_mm_loadu_pd` берёт два соседних `double`. Столбцы не участвуют вообще — транспонирование становится не «запрещённым», а попросту ненужным.
- **Нет горизонтальных операций.** Скалярное произведение в векторных регистрах потребовало бы горизонтального сложения (`haddpd`) или шаффлов, а они медленные и существенно длиннее по латентности. Здесь всё поэлементно и вертикально.
- **Пять независимых аккумуляторов.** Сложение с плавающей точкой имеет латентность в несколько тактов, и последовательная цепочка накопления упёрлась бы именно в неё, а не в пропускную способность. Пять параллельных цепочек позволяют конвейеру работать без простоев.
- **Аккумуляторы живут в регистрах весь цикл по *k***, поэтому строка результата не перечитывается из памяти десять раз.

В самом документе Intel пример дан для матриц 6×6 и вынужденно разбит на части 4 + 2, поскольку 6 не делится на 4 дорожки типа `float`. У нас 10 элементов `double` дают ровно 5 векторов — деление точное, остаточного скалярного цикла не требуется. Это удачное совпадение заданного размера блока с шириной регистра.

```
for (int inner_k = 0; inner_k < BLOCK_COLS; ++inner_k) {
    const __m128d left_broadcast =
        _mm_set1_pd(left_block[inner_row * BLOCK_COLS + inner_k]);
    const matrix_element *restrict right_row = right_block + inner_k * BLOCK_COLS;

    accumulator_0 = _mm_add_pd(accumulator_0,
        _mm_mul_pd(left_broadcast, _mm_loadu_pd(right_row + 0 * VECTOR_LANES)));
    accumulator_1 = _mm_add_pd(accumulator_1,
        _mm_mul_pd(left_broadcast, _mm_loadu_pd(right_row + 1 * VECTOR_LANES)));
    /* ... accumulator_2, accumulator_3, accumulator_4 ... */
}
```

ЧТО ИЗ ЭТОГО ПОЛУЧИЛОСЬ В МАШИННОМ КОДЕ

Компилятор распознал пару «броадкаст + умножение» и свернул её в одну инструкцию ARM64 `fmul.2d v9, v9, v31[0]` — умножение вектора на *выбранную дорожку* другого регистра. Отдельная инструкция `dup` для размножения скаляра не понадобилась. Это ровно тот эффект, ради которого схема и построена.

8 Побитовая идентичность C1 и C2

> Требование «C1 и C2 должны ПОЛНОСТЬЮ совпадать» — не формальность. Оно жёстко ограничивает реализацию, и понимание этого ограничения — главный теоретический вопрос работы.

Неассоциативность сложения с плавающей точкой

Формат IEEE 754 `double`: 1 бит знака, 11 бит порядка, 52 бита мантиссы плюс один неявный — итого 53 значащих бита. Целые числа представимы точно до 2^{53}.

Каждая операция округляет результат до 53 бит. Поэтому сложение **неассоциативно**:

$$(a + b) + c \neq a + (b + c)$$

в общем случае. Отсюда прямое следствие: **любая перестановка порядка накопления меняет биты результата**. Это значит, что в SSE2-версии нельзя:

- переставлять циклы ради скорости;
- разбивать накопление на частичные суммы и складывать их в конце;
- включать `-ffast-math` или `-Ofast` — они прямо разрешают компилятору реассоциировать выражения.

Запрет FMA

Инструкция **FMA** (fused multiply-add) вычисляет $a \cdot b + c$ с **одним** округлением: промежуточное произведение не округляется. Обычная пара «умножение, затем сложение» округляет дважды. Результаты различаются в младших битах.

Это исторически согласуется с самим заданием: в SSE2 инструкции FMA **нет** — она появилась только в FMA3/AVX2 на x86 и в NEON на ARM. Но компилятор на ARM64 с радостью свернёт `mul` + `add` в `fmla`, если ему не запретить. Поэтому в обоих ядрах стоит:

```
#pragma STDC FP_CONTRACT OFF
```

а в `Makefile` намеренно отсутствует `-ffast-math`.

Три уровня защиты

УРОВЕНЬ	ЧТО ГАРАНТИРУЕТ
Одинаковый порядок накопления	Оба ядра идут по *inner_k* по возрастанию, выполняя отдельное умножение и отдельное сложение. Каждая SIMD-дорожка независимо повторяет ровно ту последовательность операций, которую выполняет скаляр.

УРОВЕНЬ	ЧТО ГАРАНТИРУЕТ
Отсутствие FMA	`FP_CONTRACT OFF` плюс отсутствие `-ffast-math`. Подтверждено дизассемблером: ни одной инструкции `fmla` / `fmadd` ни в одном из трёх объектных файлов.
Точные входные данные	Матрицы заполняются целыми числами из диапазона [–8, +8]. Сумма 2000 произведений таких чисел по модулю не превосходит 2000 · 64 = 128 000 — это на много порядков меньше $2^{53} \approx 9{,}0 \cdot 10^{15}$. **Вся арифметика точна, округления не происходит вообще.**

```
for (size_t index = 0; index < element_count; ++index) {
    state = state * FILL_LCG_MULTIPLIER + FILL_LCG_INCREMENT;

    /* Старшие биты LCG — те, что ведут себя прилично. */
    const unsigned bucket = (state >> FILL_LCG_SHIFT) % (unsigned)FILL_VALUE_RANGE;
    matrix->elements[index] = (matrix_element)((int)bucket - FILL_VALUE_BOUND);
}
```

ПОЧЕМУ ТРЕТИЙ УРОВЕНЬ ВАЖЕН

Первые два уровня делают совпадение *гарантированным по построению*, третий делает его *гарантированным независимо от построения*. Благодаря точности данных `memcmp` — честная проверка правильности алгоритма, а не проверка того, что двум реализациям «повезло» округлиться одинаково.

Если бы преподаватель спросил, что будет на произвольных вещественных данных, корректный ответ: побитовое совпадение тогда держится исключительно на идентичности порядка операций и отсутствии FMA — и обеспечивать его нужно сознательно.

Сама проверка предельно проста и не допускает послаблений:

```
bool block_matrix_equals(const block_matrix *left, const block_matrix *right)
{
    /* Побитово по построению: никакого epsilon. */
    return left->block_rows == right->block_rows
        && left->block_cols == right->block_cols
        && memcmp(left->elements, right->elements, bytes) == 0;
}
```

9 Измерение времени

Почему запрещены clock() и time()

ФУНКЦИЯ	ПОЧЕМУ НЕПРИГОДНА
`time()`	Разрешение — одна секунда. Для измерения ядра бесполезна.

ФУНКЦИЯ	ПОЧЕМУ НЕПРИГОДНА
`clock()`	Возвращает процессорное время в единицах `CLOCKS_PER_SEC`. Номинально это 10^6, но реальная гранулярность обновления — порядка 10 мс (квант планировщика). Кроме того, семантика различается между системами: на Windows это астрономическое время, на POSIX — процессорное, а при многопоточности оно суммирует время всех потоков.

Что используется

Основное измерение — `clock_gettime(CLOCK_MONOTONIC_RAW)`, сишный эквивалент `std::chrono::high_resolution_clock`: наносекундное разрешение, монотонность (часы не прыгают при переводе системного времени), а суффикс `_RAW` дополнительно исключает подстройку со стороны NTP.

Счётчик тактов. На x86 это `rdtsc`, результат в паре регистров `edx:eax` (старшие 32 бита в EDX, младшие в EAX), интринсик `__rdtsc()`. Тонкости, за которые обычно и цепляются на защите:

- `rdtsc` **не сериализующая инструкция.** Внеочередное ядро может выполнить её до или после измеряемого кода. Лечится инструкцией `cpuid` перед ней (сериализует, но дорога и имеет переменную латентность — в примерах Intel это отдельно оговорено), либо `lfence`, либо инструкцией `rdtscp`, которая дожидается завершения предыдущих операций.
- **Constant / Invariant TSC.** На современных процессорах счётчик тикает с фиксированной номинальной частотой независимо от turbo boost и энергосберегающих состояний. Следовательно, `rdtsc` *больше не считает такты ядра* — он считает фиксированный временной базис. Пересчитывать его в «такты на элемент» при плавающей частоте некорректно.
- На многосокетных системах счётчики разных сокетов могут быть не синхронизированы, и миграция потока между ядрами портит измерение.

На ARM64 инструкции `rdtsc` нет; её роль играет системный счётчик `CNTVCT_EL0`, а его частота доступна в `CNTFRQ_EL0`. Реализация:

```
#if defined(__aarch64__)
    uint64_t ticks;
    /* isb опустошает конвейер перед чтением счётчика: без неё ядро может прочитать
       cntvct_el0 спекулятивно, вне порядка относительно измеряемого кода.
       "memory" запрещает такую же перестановку уже на уровне компилятора. */
    __asm__ __volatile__("isb\n\tmrs %0, cntvct_el0" : "=r"(ticks) : : "memory");
    return ticks;
#else
    _mm_lfence();              /* та же причина: барьер против переупорядочивания */
    return (uint64_t)__rdtsc();
#endif
```

ЛОВУШКА АССЕМБЛЕРА AARCH64

Две инструкции обязаны разделяться переводом строки, а не символом `;`: ассемблер AArch64 трактует точку с запятой как начало комментария. Запись `"isb; mrs %0, cntvct_el0"` молча собралась бы в одну лишь `isb`, и чтение счётчика исчезло бы *без единого предупреждения*.

На Apple silicon `cntvct_el0` тикает с фиксированной частотой системного таймера (на этой машине `cntfrq_el0` сообщает 1 ГГц, на более ранних моделях — около 24 МГц), а не с частотой ядра. Поэтому программа печатает и источник счётчика, и его частоту, а тики честно называет «counter ticks», а не

«тактами ядра».

Методика замера

- Только **Release** (`-O3`). В отладочной сборке нет ни встраивания, ни регистровой аллокации — измерять там бессмысленно.
- Прогрев кэшей и частоты перед замером.
- Несколько прогонов; брать **медиану или минимум**, но не среднее: выбросы от планировщика односторонние, только вверх.
- Следить, чтобы компилятор не выбросил измеряемый код как мёртвый. Здесь результат потребляется последующим сравнением, поэтому устранить его нельзя.

10 Устройство проекта и разбор кода

Код разложен по каталогам, каждый из которых отвечает за одну подсистему; заголовки подключаются по коротким именам через `-Iconfig -Imatrix -Imultiplying -Itimer -Ivendor`, объектные файлы собираются в `build/`.

ФАЙЛ	НАЗНАЧЕНИЕ
`config/config.h`	Единственный источник размеров: тип элемента, размер блока, внешние размеры, число дорожек, выравнивание. Магических чисел в коде нет.
`matrix/matrix.{h,c}`	Тип `block_matrix`, выделение выровненной памяти, детерминированное заполнение, побитовое сравнение.
`multiplying/multiply.h`	Контракт трёх функций умножения, включая требование одинакового порядка накопления.
`multiplying/multiply_plain_kernel.h`	Общее скалярное ядро — один `static inline` на оба обычных варианта.
`multiplying/multiply_auto.c`	C1: то же ядро, обычные флаги, компилятор векторизует сам.
`multiplying/multiply_scalar.c`	База сравнения: то же ядро, векторизация отключена флагами сборки.
`multiplying/multiply_sse2.c`	C2: ручное ядро на интринсиках SSE2.
`timer/timer.{h,c}`	Часы и счётчик тактов, без `clock()` и `time()`.
`main.c`	Драйвер: конфигурация, три прогона, проверки, таблица результатов, код возврата.
`vendor/sse2neon.h`	Вендорится как есть, не редактируется.

Приём: один исходник — две сборки

Требование «доказать, что версия без векторизации действительно без векторизации» решается так, что доказывать почти ничего не приходится. Оба обычных варианта включают **один и тот же заголовок** и вызывают **одну и ту же функцию**:

multiply_auto.c

```
#include "multiply_plain_kernel.h"

void multiply_blocks_auto(...)
{
    multiply_blocks_plain(left, right,
result);
}
```

multiply_scalar.c

```
#include "multiply_plain_kernel.h"

void multiply_blocks_scalar(...)
{
    multiply_blocks_plain(left, right,
result);
}
```

Различаются они исключительно правилом сборки — векторизация отключается на уровне **единицы трансляции**, а не отдельного цикла:

```
NOVEC_CFLAGS = -fno-vectorize -fno-slp-vectorize -fno-tree-vectorize

$(BUILD_DIR)/multiply_scalar.o: multiplying/multiply_scalar.c | $(BUILD_DIR)
	$(CC) $(CFLAGS) $(NOVEC_CFLAGS) -c -o $@ $<
```

Тем самым сравнение честно: ускорение относительно базы — это эффект *ровно* векторизации, а не побочный результат иначе написанного кода.

Ядро на простом C

```
static inline void accumulate_block_product(
        const matrix_element *const restrict left_block,
        const matrix_element *const restrict right_block,
        matrix_element *const restrict result_block)
{
    for (int inner_row = 0; inner_row < BLOCK_ROWS; ++inner_row) {
        for (int inner_k = 0; inner_k < BLOCK_COLS; ++inner_k) {
            const matrix_element left_value = left_block[inner_row * BLOCK_COLS + inner_k];

            accumulate_scaled_row(result_block + inner_row * BLOCK_COLS,
                                  right_block + inner_k  * BLOCK_COLS,
                                  left_value);
        }
    }
}
```

Обратите внимание: это **та же самая схема**, что и в SSE2-ядре — скаляр из A умножается на строку B, — только выраженная обычным циклом. Именно поэтому автовекторизатор способен превратить её в тот же машинный код, и именно поэтому порядок накопления у двух реализаций совпадает без дополнительных усилий.

Контракты, вынесенные в заголовки

Порядок накопления

Зафиксирован в `multiply.h` как часть контракта всех трёх функций. Его нельзя менять ради скорости — это вопрос корректности, а не оптимизации.

Очистка результата

Функции *накапливают* в результат, но не очищают его. Очистка — обязанность вызывающего, и это же условие делает законным применение `restrict`.

Проверки на этапе компиляции

`_Static_assert` проверяет, что дорожек ровно две и что строка блока делится на них нацело. Нарушение размера блока обрушит сборку, а не даст молча неверный результат.

Предусловия

`assert` на согласованность размерностей стоит перед циклами, а не внутри них, — на горячий путь не влияет.

Что печатает программа

Вывод фрагментов матриц запрещён, поэтому печатаются только конфигурация, вердикты проверок и измерения. Код возврата — `EXIT_SUCCESS` только если прошли все проверки, то есть результат работы пригоден для автоматической проверки.

11 Результаты измерений

Измерения выполнены на Apple M5, сборка `-O3 -march=native`, тип `double`, блок 10 × 10, внешний размер 400 × 400 блоков (плоский размер 4000 × 4000, 122 MiB на матрицу). Общий объём вычислений — $1{,}28 \cdot 10^{11}$ операций с плавающей точкой. Каждая версия прогоняется три раза, в таблицу идёт самый быстрый прогон.

ВЕРСИЯ	ВРЕМЯ, МС	ТИКИ СЧЁТЧИКА	GFLOP/S	УСКОРЕНИЕ
скалярная (векторизация отключена)	7430,917	7 430 919 721	17,225	1,00×
автовекторизованная — **C1**	3814,617	3 814 619 981	33,555	1,95×
интринсики SSE2 — C2	**3769,163**	**3 769 166 247**	**33,960**	**1,97×**

C1 = C2, побитово

PASS — `memcmp` по всем 16 000 000 элементов

база = C1

PASS — отключение векторизации не меняет результат

C2 не медленнее C1

PASS — 3769 мс против 3815 мс, в 5/5 прогонов

Инструкций FMA

0 — во всех трёх объектных файлах

Интерпретация

- **Ускорение 1,97×** при теоретическом потолке 2,00× для `double` на 128-битном регистре (две дорожки). Это 98,5 % от предела — векторная версия использует SIMD практически со стопроцентной эффективностью.
- Методичка называет ориентиром для `double` 1,2–1,5×. Превышение объясняется тем, что база сравнения здесь — *тот же самый исходный код* с отключённой векторизацией, а не иначе написанный скалярный вариант; никаких посторонних различий в сравнение не попадает.
- **Ручная версия стабильно быстрее автоматической** (3769 против 3815 мс, около 1,2 %; в серии из пяти независимых прогонов SSE2 выигрывала 5 раз из 5). Причина видна в дизассемблере: автовекторизатор распределил накопление по регистрам менее аккуратно, тогда как в ручной версии пять аккумуляторов зафиксированы явно и живут в регистрах весь внутренний цикл. Требование «SSE2 не медленнее» выполнено.
- **17,2 GFLOP/s у скалярной версии** — это уже очень высокий результат для невекторизованного кода, и он целиком заслуга блочной раскладки: данные приходят из L1, а не из DRAM. Именно поэтому векторизация здесь даёт чистые 2×: упираться больше не во что.

КАК ПОДБИРАЛСЯ ВНЕШНИЙ РАЗМЕР

Методичка требует, чтобы построение C1 занимало «от нескольких секунд». Первоначальная калибровка на черновом ядре давала 4,8 GFLOP/s и подсказывала размер 200 × 200 блоков; готовый код оказался в семь раз быстрее, и при этом размере C1 считалась всего 0,48 с. Поскольку время растёт как куб внешнего размера, переход на 400 × 400 блоков дал ровно нужные **3,81 с** при 4000 × 4000 элементах и 122 МБ на матрицу. Соотношения между версиями при этом не изменились, что само по себе подтверждает: измеряется свойство кода, а не случайный эффект конкретного размера.

12 Доказательство векторизации

Три способа, по возрастанию убедительности.

Способ 1. Отчёты компилятора

Цель `make report` пересобирает оба обычных ядра с включёнными замечаниями оптимизатора (`-Rpass=loop-vectorize -Rpass-missed=loop-vectorize`). Результат оказался неожиданным:

Обычная сборка `multiply_auto.c`:

```
multiply_plain_kernel.h:53:5: remark: the cost-model indicates that
                              vectorization is not beneficial [-Rpass-missed=loop-vectorize]
multiply_plain_kernel.h:44:31: remark: Stores SLP vectorized with cost -32
                               and with tree size 43 [-Rpass=slp-vectorizer]
   44 |  result_row[inner_col] += left_value * right_row[inner_col];
```

Скалярная сборка того же исходника с `NOVEC_CFLAGS`:

```
multiply_plain_kernel.h:53:5: remark: loop not vectorized [-Rpass-missed=loop-vectorize]
(замечания SLP нет вообще)
```

Компилятор прямо сообщает, что произошло: loop-векторизатор **отказался** от цикла («невыгодно»), а векторный код породил **SLP-векторизатор** — с оценкой выигрыша −32, то есть он счёл преобразование существенно выгодным.

Способ 2. Дизассемблер

Самое прямое доказательство. На ARM64 суффикс `.2d` означает «две упакованные величины двойной точности»; скалярные операции работают с регистрами `d0…d31`.

ОБЪЕКТНЫЙ ФАЙЛ	ВЕКТОРНЫХ (.2D)	СКАЛЯРНЫХ (D)	FMA	ВЫВОД
`multiply_auto.o`	100	0	0	ВЕКТОРИЗОВАН
`multiply_scalar.o`	0	200	0	СКАЛЯРНЫЙ
`multiply_sse2.o`	100	0	0	ВЕКТОРИЗОВАН

Соотношение 100 против 200 говорит само за себя: векторная версия выполняет ровно вдвое меньше инструкций для того же объёма арифметики — по две величины за инструкцию.

multiply_sse2.o — векторно

multiply_scalar.o — скалярно

```
fmul.2d  v9,  v9, v31[0]
fadd.2d  v26, v26, v9
fmul.2d  v9,  v9, v31[0]
fadd.2d  v27, v27, v9
```

```
fmul  d16, d16, d6
fadd  d4,  d4,  d16
fmul  d16, d16, d6
fmul  d17, d17, d6
```

Заодно это доказывает отсутствие FMA: везде `fmul` и `fadd` раздельно, ни одной `fmla`. Именно на этом держится побитовое совпадение C1 и C2.

Способ 3. Прямой эксперимент: какой векторизатор виноват

Методичка выносит в отдельный пункт вопрос «как отключить векторизацию???». Ответ получен экспериментально: один и тот же файл `multiply_auto.c` собран с разными комбинациями флагов.

ФЛАГИ	ВЕКТОРНЫХ (.2D)	СКАЛЯРНЫХ (D)	ИТОГ
`-O3` (по умолчанию)	100	0	векторизовано
`-fno-vectorize`	100	0	БЕЗ ЭФФЕКТА
`-fno-slp-vectorize`	0	200	СРАБОТАЛО
`-fno-vectorize -fno-slp-vectorize`	**0**	**200**	**используется в Makefile**

КЛЮЧЕВОЙ ВЫВОД

Весь векторный код в автоматической версии порождён **SLP-векторизатором**, а не loop-векторизатором. Флаг `-fno-vectorize`, который интуитивно кажется ответом на вопрос методички, **не даёт ровно ничего**.

Механизм понятен: внутренний цикл короткий и известной длины (10 итераций), компилятор полностью разворачивает его ещё до векторизации, после чего loop-векторизатору работать уже не над чем — цикла нет. Зато остаётся линейная последовательность независимых умножений и сложений, и её подхватывает SLP-векторизатор, склеивая соседние операции в векторные. Стоимостная модель loop-векторизатора при этом честно сообщает, что векторизация «невыгодна», — потому что к моменту её вызова работа уже сделана другим проходом.

Практический вывод для любого, кто будет отключать векторизацию для замеров: **гасить нужно оба векторизатора**. Отключение одного создаёт убедительную иллюзию скалярной сборки, которая на деле остаётся векторной, и все последующие измерения ускорения оказываются недействительными.

13 Особенности решения

1. SSE2-код, исполняемый на ARM64

Ядро написано на буквальных интринсиках SSE2 и транслируется в NEON на этапе компиляции, без рантайм-слоя и без потери производительности. Тот же файл соберётся на x86 заменой одной строки `#include`. Платформенная зависимость локализована в шапке из шести строк.

2. Отключение векторизации на уровне единицы трансляции

Скалярная база и автоматическая версия собираются из *физически одного и того же* исходника, различаясь только флагами. Сравнение измеряет эффект ровно векторизации, а не разницу между двумя по-разному написанными программами.

3. Побитовое равенство обеспечено, а не найдено

Одинаковый порядок накопления, явный запрет FMA и точные целочисленные данные. `memcmp` вместо сравнения с допуском — и это обосновано, а не рискованно.

4. Блочная раскладка памяти

Блоки хранятся непрерывно, поэтому блочность из условия задачи бесплатно превращается в тайлинг по Drepper'у: полное использование кэш-линий, минимум страниц, работающая предвыборка.

5. Выявлена роль SLP-векторизатора

Экспериментально показано, что `-fno-vectorize` на этом коде бесполезен и что весь векторный код порождает SLP-проход. Это прямой и подтверждённый ответ на вопрос методички.

6. Измерение на ARM без rdtsc

Счётчик реализован через `CNTVCT_EL0` с барьером `isb`, частота читается из `CNTFRQ_EL0` и сообщается честно — тики не выдаются за такты ядра.

Эффект restrict, измеренный количественно

Побочный, но показательный результат. Первая версия ядра была написана как одно гнездо циклов с `restrict` на указателях блоков. Она давала **4,8 GFLOP/s**. Вынесение обновления строки в отдельную функцию с `restrict` на *параметрах* подняло производительность до **33 GFLOP/s** — в 6,9 раза, при полностью идентичной арифметике.

Причина в том, где именно компилятор использует это обещание. На параметрах функции `restrict` позволяет доказать независимость итераций внутреннего цикла напрямую; на указателях внешнего гнезда, прошедших через несколько уровней индексной арифметики, такое доказательство не выводится, и векторизация не происходит. Это наглядная иллюстрация того тезиса Drepper'а, что правила алиасинга C — главный практический барьер для оптимизации численного кода.

14 Вопросы на защиту

Почему нельзя транспонировать и что сделано вместо этого?

Транспонирование требует дополнительной памяти под копию и самого прохода копирования. Вместо него применена схема Intel AP-930: элемент A размножается по дорожкам вектора и умножается на целую строку B. Строка читается вперёд с шагом 1, столбцы не участвуют — транспонирование становится ненужным.

Почему для double ускорение меньше, чем для float?

В 128-битный регистр помещается две величины двойной точности против четырёх одинарной. Потолок — 2× против 4×. Получено ровно 2,00×, то есть предел достигнут.

Почему блок именно такого размера помогает?

Блок 10 × 10 занимает 800 байт, пара блоков гарантированно помещается в L1d объёмом 64 KiB. Блок хранится непрерывно, поэтому кэш-линии по 128 байт выбираются полностью, а не на 62 %, и блок занимает одну страницу вместо десяти.

Как доказано, что версия векторизована?

Дизассемблером: 100 инструкций с суффиксом `.2d` против 200 скалярных на регистрах `d` — ровно вдвое меньше инструкций на тот же объём арифметики. Дополнительно — замечания оптимизатора `-Rpass` и контрольный эксперимент с флагами.

Как отключена векторизация?

На уровне единицы трансляции: одно и то же ядро собирается дважды, для скалярной версии добавляются `-fno-vectorize -fno-slp-vectorize`. Важно, что первого флага недостаточно — весь векторный код здесь порождает SLP-векторизатор, что подтверждено экспериментом.

Почему C1 и C2 совпали побитово, ведь сложение неассоциативно?

Три причины: одинаковый порядок накопления в обоих ядрах, явный запрет FMA (`FP_CONTRACT OFF` и отсутствие `-ffast-math`), и целочисленные данные из диапазона [−8, 8], для которых вся арифметика точна — накопленная сумма не превосходит 128 000 при границе точности 2^{53}.

Почему не используется FMA, он же быстрее?

FMA округляет один раз вместо двух и потому даёт другой результат, а задание требует полного совпадения C1 и C2. Кроме того, в самом SSE2 инструкции FMA нет — она появилась только в FMA3/AVX2.

Почему запрещён clock()?

Реальная гранулярность около 10 мс независимо от номинального `CLOCKS_PER_SEC`, плюс различная семантика между операционными системами и суммирование времени всех потоков при многопоточности.

Что такое rdtsc и где оказывается результат?

Счётчик меток времени процессора; результат возвращается в паре регистров `edx:eax`. Инструкция не сериализована, поэтому нужен `rdtscp` либо барьер `lfence`. На современных процессорах счётчик тикает с постоянной частотой и потому не равен тактам ядра.

> **Где узкие места по производительности?**
>
> Код упирается в вычисления, а не в память: блочная раскладка обеспечивает подачу данных из L1, требуемая полоса DRAM — около 13 ГБ/с при доступных десятках. Ограничителями остаются отсутствие FMA (сознательное, ради побитового совпадения) и длина цепочки зависимостей при накоплении. Дальнейшие шаги — второй уровень блокировки под L2 и упаковка панелей, то есть подход GotoBLAS.

> **Почему скалярная версия уже даёт 17 GFLOP/s — это не слишком много?**
>
> Это заслуга блочной раскладки: даже без SIMD данные приходят из кэша первого уровня, и процессор не простаивает. Именно поэтому векторизация затем даёт чистые 2× — устранять больше нечего, кроме числа величин на инструкцию.

Источники

- Intel. *AP-930: Streaming SIMD Extensions — Matrix Multiplication*, Order 245045-001, 1999. — схема broadcast-and-accumulate.
- U. Drepper. *What Every Programmer Should Know About Memory*, 2007. — §6.2.1, оптимизация доступа к L1d, таблица 6.2.
- A. Krivutsenko. *GotoBLAS — Anatomy of a Fast Matrix Multiplication*, TU München, 2008. — многоуровневая блокировка и упаковка панелей.